

Theater Ticketing System

Software Requirements Specification

Version 2.0

July 21, 2026

Group CS250-26

Pranith Bollampally

Prepared for

CS 250 – Introduction to Software Systems (Section 02)

Instructor: Gus Hanna, Ph.D.

Summer 2026

Revision History

Date	Description	Author	Comments
07/14/2026	Version 1.0	Pranith Bollampally	First revision — requirements specification
07/21/2026	Version 2.0	Pranith Bollampally	Added Section 6, Software Design (architecture, UML class diagram, development plan)

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
	Pranith Bollampally	Software Eng.	
	Dr. Gus Hanna	Instructor, CS 250	

Table of Contents

Revision History 2

Document Approval..... 2

Table of Contents 2

1. Introduction 5

 1.1 Purpose..... 5

 1.2 Scope 5

 1.3 Definitions, Acronyms, and Abbreviations..... 5

 1.4 References 5

 1.5 Overview 6

2. General Description 6

 2.1 Product Perspective..... 6

 2.2 Product Functions..... 6

 2.3 User Characteristics 6

 2.4 General Constraints 6

 2.5 Assumptions and Dependencies..... 7

 2.6 User Requirements 7

3. Specific Requirements 7

 3.1 External Interface Requirements..... 7

 3.1.1 User Interfaces..... 7

 3.1.2 Hardware Interfaces 7

 3.1.3 Software Interfaces 7

 3.1.4 Communications Interfaces 8

 3.2 Functional Requirements..... 8

 3.2.1 Showtime Browsing..... 8

 3.2.2 Ticket Purchase and Seat Reservation..... 8

 3.2.3 Payment Processing..... 8

 3.2.4 Discount Application..... 8

3.2.5 Customer Accounts.....	8
3.2.6 Ticket Issuance and Validation.....	9
3.2.7 Refunds.....	9
3.2.8 Administrator Mode.....	9
3.2.9 Customer Feedback.....	9
3.2.10 Bot Mitigation and Demand Queueing.....	9
3.3 Use Cases.....	9
3.3.1 Use Case #1: PurchaseTicket.....	10
3.3.2 Use Case #2: RefundTicket.....	11
3.3.3 Use Case #3: ManageShowtimes.....	12
3.4 Classes / Objects.....	12
3.5 Non-Functional Requirements	12
3.5.1 Performance	12
3.5.2 Reliability	12
3.5.3 Availability.....	13
3.5.4 Security.....	13
3.5.5 Maintainability.....	13
3.5.6 Portability.....	13
3.6 Inverse Requirements	13
3.7 Design Constraints.....	13
3.8 Logical Database Requirements	13
3.9 Other Requirements	14
4. Analysis Models.....	14
4.1 Sequence Diagrams.....	14
4.2 State-Transition Diagrams (STD).....	14
4.3 Data Flow Diagrams (DFD)	14
5. Change Management Process	14
6. Software Design.....	15
6.1 System Overview.....	15
6.2 Software Architecture.....	15
6.2.1 Architecture Diagram.....	15
6.2.2 Component Descriptions.....	15
6.3 UML Class Diagram	16
6.3.1 Class Diagram.....	16
6.3.2 Class Descriptions.....	17
6.4 Development Plan and Timeline.....	20
6.4.1 Task Partitioning and Timeline.....	21
6.4.2 Team Member Responsibilities	21
A. Appendices	22
A.1 Elicitation Sources	22

1. Introduction

This is the Software Requirements Specification (SRS) for the Theater Ticketing System (TTS), a replacement for the ticket-sales software used by a twenty-location movie theater chain in San Diego. It records what the system must do and the constraints it must respect, so that design and testing can proceed from an agreed baseline.

1.1 Purpose

This SRS captures the requirements for the TTS as requested by the client by interviewing and supporting documentation, establishing an agreement on what will be built before design begins. Its audience is the client who confirms it describes the system they asked for; the engineering team, who will base the design, test plan, and implementation on it; and the course instructor. It does not describe how the system will be built — design decisions belong in the Software Design Specification.

1.2 Scope

The product specified is the Theater Ticketing System (TTS).

What the TTS will do

The TTS is a browser-based application that lets customers browse showtimes across the chain's twenty theaters, buy up to twenty tickets at a time, reserve assigned seats in deluxe theaters, claim student, military, and senior discounts, pay by card or online payment service, and receive tickets by email or at the box office. It lets administrators set showtimes, assign films to theaters, override mistaken purchases, and see sales. It lets box office staff refund tickets in person. It collects a one-tap satisfaction rating after each purchase and shows critic scores alongside each film.

What the TTS will not do

- It will not be a native mobile app — the client explicitly asked for browser-based.
- It will not sell tickets valid outside the San Diego chain or handle non-USD currency.
- It will not offer self-service refunds; the client requires an employee in person.
- It will not recommend nearby theaters — the client deferred this to a later iteration.

Who is affected

- Moviegoers — the cinema-going public, buying from their own browser or a lobby kiosk. The largest group cannot be trained, and the group the system earns its money from.
- Box office staff — trained employees who hand out tickets and are the only people who can issue a refund, usually with a customer waiting in front of them.
- Theater managers and ownership — a small, trained group who decide what plays where, fix customer mistakes, and need to know how each film is selling. Also, the client pays for the system.

Two outside systems are involved: the payment service and the critical service. Neither is under the chain's control, and the TTS must keep working sensibly when either is unavailable.

Benefits, objectives, and goals

The client's current system sells tickets but is slow and carries defects accumulated across updates. The goals of the replacement are to serve at least 1,000 concurrent customers without degradation; deliver a user-testable prototype within four months; hold cost at or below \$500,000; show identical live-updating inventory to online and kiosk customers so a seat cannot be double-sold; and reduce both the defect load and the frequency of updates that force downtime.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
TTS	Theater Ticketing System — the product specified by this document.
Regular / Deluxe theater	Regular: 150 general-admission seats, no seat assignment. Deluxe: 75 assigned seats, one per ticket.
Moviegoer / BoxOfficeAgent / TheaterAdministrator	Actor roles: customer; employee who issues tickets and refunds in person; manager who sets showtimes, overrides, and views sales.
Reservation hold	The temporary claim on seats while a customer completes a purchase; expires after five minutes.
UR / FR / NFR	User / Functional / Non-Functional Requirement identifiers used throughout this document.

1.4 References

- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications. IEEE, 1998.

- Client interview, client requirements list, and supplementary elicitation notes (Theater Ticketing System Qs.rtf; Theater Ticketing Requirements.rtf; theatre-ticketing questions.txt; theater-ticketing-reqs.txt). CS 250 Week 2 module, SDSU, July 13, 2026.
- Arms, William Y. Scenarios and Use Cases. CS 5150, Cornell University. Distributed as lecture-use-cases.pdf, CS 250 Week 2 module. Use case format follows this source.

1.5 Overview

Section 2 gives the background needed to interpret the requirements — how the product relates to the existing system, its functions, its users, its constraints, the assumptions it rests on, and the user requirements in the users' own terms. Section 3 is the core: numbered, testable requirements covering external interfaces (3.1), functional requirements by feature (3.2), the use case diagram and three full use cases (3.3), non-functional requirements by quality attribute (3.5), and inverse, design, and database requirements (3.6–3.9). Sections 3.4 and 4 are deferred to later deliverables. Section 5 describes how this document changes. Appendix A records the elicitation sources and the conflicts between them.

Requirements are tagged UR-n, FR-n, and NFR-n. These tags are stable and are what later design and test artifacts trace back to. Priority is Essential (unacceptable without), Desirable (asked for, usable without), or Optional (if budget allows).

2. General Description

2.1 Product Perspective

The TTS replaces, rather than extends, the chain's existing ticketing software, which the client describes as meeting the bare requirement while being slow and carrying compounding defects. It is not self-contained: a single chain-wide database partitioned by theater holds all film, showtime, seat, ticket, and account data; an external gateway authorizes payments; an external service supplies critic scores; and lobby kiosks and customer browsers are two front ends onto the same inventory, which the client needs to show the same data and update live. Because both channels share one inventory in real time, the TTS — not either front end — is the single authority on availability.

2.2 Product Functions

- Present films, showtimes, and availability across twenty theaters, with critic scores alongside each film.
- Sell up to twenty tickets per transaction, from two weeks before showtime until ten minutes after it starts.
- Assign specific seats in deluxe theaters; sell general admission in regular theaters.
- Hold inventory for five minutes during a purchase, then release it.
- Apply student, military/veteran, and senior discounts, and take payment through an external gateway.
- Issue unique, non-replicable tickets by email or box office collection, printable or digital.
- Offer optional accounts with saved payment methods, purchase history, and loyalty points, and sell memberships.
- Block automated bulk buying and queue customers fairly when demanding one showing spikes.
- Support in-person refunds by an employee, and an administrator mode for showtimes, overrides, and sales reporting.
- Collect a one-tap satisfaction rating after each purchase.

2.3 User Characteristics

Moviegoer — the public. No training, no technical background, and no familiarity can be assumed; most use the system a few times a year. Kiosk users are often standing, hurried, and queued behind others, so the path from arrival to purchase must be short. Spanish support implies a meaningful Spanish-preferring share. This is the only role that cannot be trained, and it generates essentially all revenue.

BoxOfficeAgent — a trained employee using the system daily, under time pressure with a customer waiting. Needs speed on a few well-understood operations rather than breadth. Sole channel for refunds.

TheaterAdministrator — a small, trained group of managers; the only role permitted to change what the system sells, and the consumer of sales reporting. Has the most destructive potential, hence the strictest access control and audit requirements.

PaymentGateway and **ReviewAggregator** are non-human actors outside the team's control and may be slow or unavailable at any time.

2.4 General Constraints

- Budget of approximately \$500,000, with the client asking that costs be kept low.
- A user-testable prototype within four months of project starts.
- Must run in a web browser; a native mobile app is explicitly ruled out.
- Twenty theaters, all San Diego, all U.S. Pacific time, U.S. dollars only.

- Fixed physical capacity: 150 seats regular, 75 assigned seats deluxe; a theater is entirely one or the other; one film per theater at a time, about six to eight showtimes a day.
- Updates are as infrequent as possible, to minimize downtime.
- Card payments bring payment card handling obligations, hence delegation to an external gateway.

2.5 Assumptions and Dependencies

- A-1. The chain stays at twenty San Diego theaters in one time zone. NFR-1 and NFR-2 are sized against this footprint.
- A-2. A suitable payment gateway is available within budget. FR-9 and FR-27 through FR-30 depend on it.
- A-3. Critic data is obtainable under a license permitting display. The supplementary source describes scraping review sites; this specification assumes a licensed feed instead, since scraping is legally exposed and fragile. If no affordable source exists, FR-10 and FR-15 must be renegotiated.
- A-4. The client's figure of roughly 1,000 concurrent users is the accurate one. A supplementary source states 10,000,000; this document treats that as an error (see A.2). If the client confirms the higher figure, the budget, schedule, and architecture all change, and this SRS must be rewritten rather than amended.
- A-5. Discount eligibility is checked by an employee at the door, not by the system. No source describes a verification mechanism, and building one is not funded.
- A-6. Swedish support, stated in the supplementary sources, is genuinely required despite the chain being San Diego-only. Flagged for confirmation; see A.2.

2.6 User Requirements

What the system must offer from its users' point of view, in their terms. Each is realized by the functional and non-functional requirements in Section 3.

UR-1 As a Moviegoer, I can see what is playing, where and when, across all theaters, and what critics thought — without making an account.

UR-2 As a Moviegoer, I can buy up to twenty tickets at once, and I am told clearly if I try to buy more.

UR-3 As a Moviegoer, I pick my seat when the theater has assigned seating and am not asked to when it does not.

UR-4 As a Moviegoer, I am never shown a seat as available and then told it is taken after I have paid.

UR-5 As a Moviegoer, I get a student, military, or senior prize when I am entitled to one, and I can pay without an account.

UR-6 As a Moviegoer, I get my ticket by email or at the box office, printable or on my phone, and I have time to finish paying without being rushed.

UR-7 As a Moviegoer, I can use the system in English or Spanish, get a refund by speaking to someone at the theater, and am not shut out of a popular showing by someone running a bot.

UR-8 As a BoxOfficeAgent, I can find a booking and refund it while the customer stands in front of me, and the seat returns to sale immediately.

UR-9 As a TheaterAdministrator, I can set which film plays where and when, correct a customer's mistake, and see how each film sells and nobody who is not an administrator can do any of it.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

FR-1 The TTS shall present its interface through a web browser; no native app shall be required for any customer task. (Essential)

FR-2 The TTS shall present a public web interface and a lobby kiosk interface onto identical inventory, both reflecting a sale made through the other without a page reload. (Essential)

FR-3 The TTS shall present all customer-facing text in English, Spanish, and Swedish, selectable by the customer. (English/Spanish Essential; Swedish pending confirmation — see A.2)

FR-4 The TTS shall present an administrator interface reachable only by an authenticated TheaterAdministrator. (Essential)

FR-5 The TTS shall show the same price for a given ticket everywhere it appears, across both front ends. (Essential)

3.1.2 Hardware Interfaces

FR-6 The TTS shall interface with kiosk terminals, including touch input and attached ticket printers. (Essential)

FR-7 The TTS shall interface with box office printers to produce physical tickets. (Essential)

3.1.3 Software Interfaces

FR-8 The TTS shall interface with a single chain-wide database, partitioned by theater. (Essential)

FR-9 The TTS shall interface with an external payment gateway and shall not retain raw card numbers. (Essential)

FR-10 The TTS shall interface with an external review aggregation service for critic scores and quotes. (Desirable)

FR-11 The TTS shall interface with an email service to deliver tickets and confirmations. (Essential)

3.1.4 Communications Interfaces

FR-12 All traffic between browsers or kiosks and the TTS shall use HTTPS. (Essential)

FR-13 Tickets shall be delivered in a format that renders both as a digital ticket and as a printable document. (Essential)

3.2 Functional Requirements

3.2.1 Showtime Browsing

FR-14 The TTS shall display all showings in the bookable window across all twenty theaters — film, theater, showtime, type, price, availability — filtered by any criteria supplied. (Essential)

FR-15 The TTS shall display an aggregated critic score and at least one critic quote alongside each film. (Desirable)

FR-16 The TTS shall permit browsing without authentication. (Essential)

FR-17 If the ReviewAggregator is unavailable, the TTS shall display the showing without critic information rather than failing. Review data shall never be a precondition of a sale. (Essential)

3.2.2 Ticket Purchase and Seat Reservation

FR-18 The TTS shall permit a maximum of twenty tickets per transaction, rejecting larger requests with a clear statement of the limit while retaining the Moviegoer's other selections. (Essential)

FR-19 The TTS shall sell tickets for a show from two weeks before it's showtime until ten minutes after it begins, and at no other time. (Essential)

FR-20 For a deluxe theater, the TTS shall present the 75-seat layout with availability and require one specific seat per ticket. (Essential)

FR-21 For a regular theater, the TTS shall sell general admission against a capacity of 150 and shall not present a seat map. (Essential)

FR-22 On starting a purchase, the TTS shall hold the selected inventory, making it unavailable on both front ends, and shall display a five-minute countdown. (Essential)

FR-23 The TTS shall release the hold after five minutes, returning inventory to sale and the Moviegoer to selection for the same showing rather than discarding their context. (Essential)

FR-24 The TTS shall permit purchase without an account, offering but not requiring account creation, and shall permit a membership card to be applied. (Essential)

FR-25 If a seat is claimed by another Moviegoer between display and selection, the TTS shall say so, refresh the seat map, and allow reselection without restarting the transaction. (Essential)

FR-26 The TTS shall not, under any circumstance, confirm two tickets for the same assigned seat in the same showing. (Essential)

3.2.3 Payment Processing

FR-27 The TTS shall accept credit card and online payment service, priced in U.S. dollars, submitting details to the external gateway and confirming tickets only on authorization. (Essential)

FR-28 The TTS shall record every purchase and refund in a chain-wide daily log capturing ticket identifier, showing, theater, price, discount, payment method, and timestamp. (Essential)

FR-29 If payment is declined, the TTS shall inform the Moviegoer, retain the hold, and permit another method; if the gateway is unreachable, it shall neither confirm nor charge. (Essential)

FR-30 If authorization succeeds but confirmation fails, the TTS shall reverse the authorization. No Moviegoer shall ever be charged without receiving a ticket. (Essential)

3.2.4 Discount Application

FR-31 The TTS shall offer students, military/veteran, and senior discounts, at most one per ticket, visible in the total from selection through payment. (Essential)

FR-32 The TTS shall record the discount category against each ticket so it can be verified at the door and reported on. (Essential) Note: the TTS does not verify eligibility — see A-5.

3.2.5 Customer Accounts

FR-33 The TTS shall permit an optional loyalty account holding contact details, saved payment methods, purchase history, and loyalty points. (Desirable)

FR-34 The TTS shall permit only one device logged into an account at a time; a second login shall terminate the first session and release any hold it held. (Desirable)

FR-35 The TTS shall permit purchase of a membership card subscription through the system. (Desirable)

3.2.6 Ticket Issuance and Validation

FR-36 The TTS shall generate for each ticket a unique identifier that cannot be forged or duplicated, verifiable at the door as authentic and not previously admitted. (Essential)

FR-37 The TTS shall deliver tickets by email or box office collection at the Moviegoer's election, each printable and digitally displayable. (Essential)

FR-38 The TTS shall mark a ticket admitted when accepted at the door and reject any later presentation of it, flagging failures to a BoxOfficeAgent. (Essential)

FR-39 If email delivery fails, the ticket shall remain valid and collectable at the box office, and the TTS shall say so on screen. Delivery failure shall never void a paid ticket. (Essential)

3.2.7 Refunds

FR-40 The TTS shall permit only an authenticated BoxOfficeAgent or TheaterAdministrator to refund a ticket, and only before its showtime has begun. (Essential)

FR-41 On refund, the TTS shall void the ticket, restore the seat on both front ends immediately, submit a gateway reversal, and log the refund with the agent's identity. (Essential)

FR-42 If the ticket is not found, already refunded, or already admitted, the TTS shall decline and state which condition applies. (Essential)

FR-43 If the reversal fails at the gateway, the TTS shall still void the ticket and restore inventory and flag the failed reversal for manual resolution rather than discarding it. (Essential)

3.2.8 Administrator Mode

FR-44 The TTS shall permit an authenticated TheaterAdministrator to create, modify, and remove showtimes (about six to eight per theater per day) and to assign a film to a theater. (Essential)

FR-45 The TTS shall permit an authenticated TheaterAdministrator to override a customer's mistaken purchase, changing its showing, seat, or ticket category. (Essential)

FR-46 The TTS shall report tickets sold per film and revenue per ticket to an authenticated TheaterAdministrator. (Essential)

FR-47 The TTS shall log every administrator's action (who, what, affected records, timestamp) and reject and log any such action from an unauthenticated session. (Essential)

FR-48 The TTS shall reject removing or reassigning a showing with tickets sold unless explicitly confirmed, identifying every affected ticket holder on confirmation. (Essential)

3.2.9 Customer Feedback

FR-49 The TTS shall present a range of smiley-face ratings after each completed purchase, record the selection, and make aggregated ratings available to a TheaterAdministrator. (Desirable)

FR-50 The TTS shall permit the prompt to be dismissed. Feedback shall never be a precondition of receiving a ticket, and a failure to record it shall never affect the completed purchase. (Desirable)

3.2.10 Bot Mitigation and Demand Queueing

FR-51 The TTS shall detect and block automated agents attempting bulk purchase of high-demand showings, while providing any blocked requester a means to demonstrate they are human. (Essential)

FR-52 When demand for a showing exceeds capacity, the TTS shall queue customers in arrival order, show each their position, and release an absent customer's place at their turn. (Essential)

FR-53 The TTS shall enforce the twenty-ticket limit server-side, so it cannot be bypassed by a client that avoids the user interface. (Essential)

3.3 Use Cases

Following the course lecture notes, an actor is a role rather than an individual, and each actor is a beneficiary of the use cases it connects to — the Moviegoer is the primary actor on RefundTicket because the refund is for their benefit, with the BoxOfficeAgent as the secondary actor who performs it. External systems appear as non-human actors.

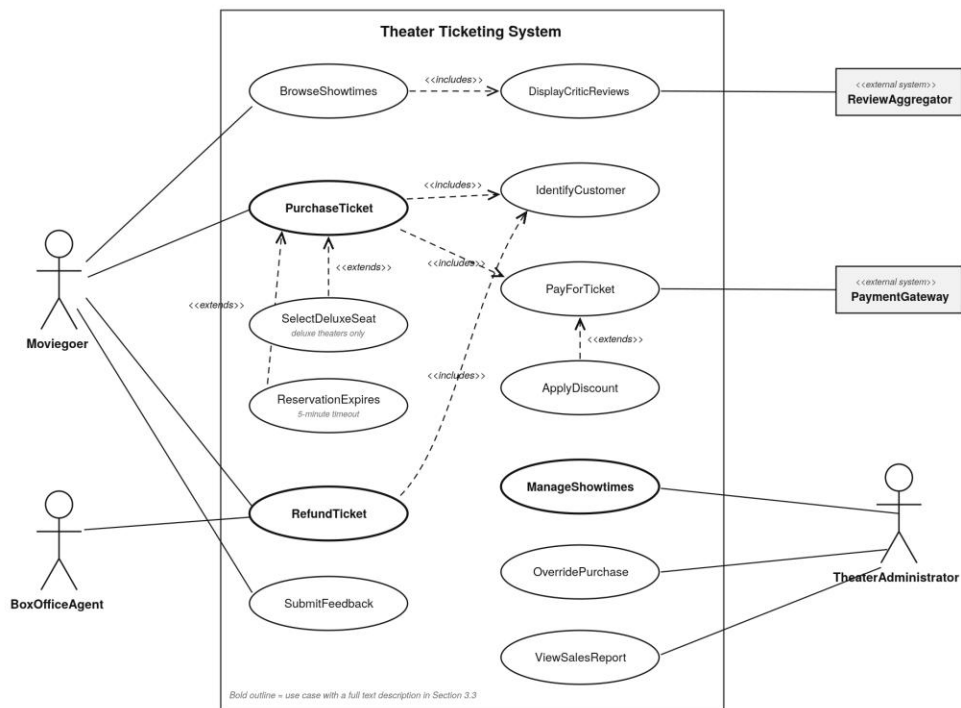


Figure 1. Theater Ticketing System — Use Case Diagram

Every use case in Figure 1 is described below. The three with a bold outline are then specified in full in 3.3.1–3.3.3.

Use case	Actor(s)	Description	Detailed
BrowseShowtimes	Moviegoer	Finds what is playing, where, when, at what price, with how many seats are left, across all twenty theaters. No account needed.	—
DisplayCriticReviews	Moviegoer, ReviewAggregator	Shows critic score and quotes while the Moviegoer decides. Included in BrowseShowtimes.	—
PurchaseTicket	Moviegoer, PaymentGateway	Buys up to twenty tickets, delivered by email or box office; inventory decrements on both front ends.	§ 3.3.1
IdentifyCustomer	Moviegoer	Registers, logs in, or proceeds without an account. Included in PurchaseTicket and RefundTicket.	—
PayForTicket	Moviegoer, PaymentGateway	Takes payment through the external gateway; confirms tickets only on authorization. Included in PurchaseTicket.	—
SelectDeluxeSeat	Moviegoer	Picks a specific seat from the 75-seat layout. Extends PurchaseTicket; deluxe theaters only.	—
ApplyDiscount	Moviegoer	Applies student, military/veterans, or senior pricing. Extends PayForTicket; only when claimed.	—
ReservationExpires	Moviegoer	The five-minute hold lapses and seats return to sale. Extends PurchaseTicket; can occur at any point in it.	—
RefundTicket	Moviegoer, BoxOfficeAgent, PaymentGateway	Refund in person from an employee before showtime; ticket voided, seat back on sale immediately.	§ 3.3.2
SubmitFeedback	Moviegoer	Taps a smiley face after purchase. Always dismissible.	—
ManageShowtimes	TheaterAdministrator	Assigns a film to a theater and sets its six to eight daily showtimes; changes go on sale immediately.	§ 3.3.3
OverridePurchase	TheaterAdministrator	Corrects a customer's mistaken purchase. Every override is audited.	—
ViewSalesReport	TheaterAdministrator	Show tickets are sold per film and revenue per ticket.	—

3.3.1 Use Case #1: PurchaseTicket

Name: PurchaseTicket — Actors: Moviegoer (primary); PaymentGateway (secondary, external) Includes IdentifyCustomer, PayForTicket; extended by SelectDeluxeSeat and ReservationExpires — Traces to UR-2 through UR-6; FR-18 through FR-26.

Flow of events

1. Moviegoer selects a showing from the browse listing.
2. TTS confirms the showing is within its bookable window — no earlier than two weeks before, no later than ten minutes after showtime.
3. Moviegoer specifies a quantity, up to twenty.
4. If the theater is deluxe, TTS invokes SelectDeluxeSeat and Moviegoer picks one seat per ticket; if regular, TTS sells general admission and no seat is selected.
5. TTS holds the inventory, removing it from availability on both front ends, and starts a visible five-minute countdown.
6. Moviegoer optionally claims a student, military/veteran, or senior discount; TTS recalculates and displays the total.
7. TTS runs IdentifyCustomer: Moviegoer authenticates, registers, or continues without an account.
8. TTS runs PayForTicket: Moviegoer supplies a payment method, and TTS submits it to PaymentGateway.
9. On authorization, TTS confirms the tickets, converts the hold to sold, generates a unique non-replicable identifier per ticket, and writes the transaction to the daily log.
10. TTS delivers tickets by email or box office collection, printable and digitally displayable, then presents the feedback prompt.

Entry conditions

1. A showing exists in the bookable window with at least one seat available.
2. Moviegoer has a supported browser or is at a kiosk. No account is required.

Exit conditions

1. Success: tickets confirmed, paid, uniquely identified, delivered, logged; inventory decremented on both front ends.
2. Failure: no charge, no ticket, hold released back to inventory.

Alternative and exceptional flows

- Step 3 — more than twenty requested: TTS rejects, states the limit, retains other selections (FR-18).
- Step 4 — seat claimed in the interim: TTS refreshes the map and allows reselection without restarting (FR-25).
- Steps 5–8 — ReservationExpires: hold lapses, inventory returns to sale, Moviegoer returns to selection (FR-23).
- Step 8 — payment declined or gateway unreachable: TTS neither confirms nor charges and permits another method within the window (FR-29).
- Step 9 — confirmation fails after authorization: TTS reverses it; nobody is charged without a ticket (FR-30).
- Step 10 — email fails: ticket stays valid and collectable at the box office (FR-39).
- Any step — request judged automated: TTS blocks it but offers a way to prove the requester is human (FR-51).

3.3.2 Use Case #2: RefundTicket

Name: RefundTicket — Actors: Moviegoer (primary — the beneficiary); BoxOfficeAgent (secondary — performs it); PaymentGateway (secondary, external) — Includes IdentifyCustomer — Traces to UR-7, UR-8; FR-40 through FR-43.

Flow of events

1. Moviegoer presents a ticket or booking reference to a BoxOfficeAgent and requests a refund.
2. BoxOfficeAgent authenticates to the TTS if not already authenticated for the shift.
3. BoxOfficeAgent enters the reference; TTS runs IdentifyCustomer to locate the booking and confirm it exists, is unrefunded, and is unadmitted.
4. TTS displays the booking: film, theater, showtime, seat, price paid, discount, payment method.
5. TTS confirms the showtime has not begun and proceeds only if it has not.
6. BoxOfficeAgent confirms the refund.
7. TTS voids the ticket so it can no longer be admitted.
8. TTS returns the seat to available inventory immediately, on both front ends.
9. TTS submits a reversal to the PaymentGateway and logs the refund with the agent's identity.
10. TTS confirms to the BoxOfficeAgent, who informs the Moviegoer.

Entry conditions

1. The Moviegoer holds a ticket bought through the TTS and is physically at a theater.
2. The BoxOfficeAgent holds valid BoxOfficeAgent or TheaterAdministrator credentials. No other role may be refunded.

Exit conditions

1. Success: ticket void, inventory back on sale, reversal submitted, refund logged and attributed.

2. Failure: ticket remains valid, no reversal submitted.

Alternative and exceptional flows

- Step 3 — not found, already refunded, or already admitted: TTS declines and states which (FR-42).
- Step 5 — showtime has begun: TTS declines the refund (FR-40).
- Step 9 — reversal fails: TTS still voids and restores inventory and flags the reversal for manual resolution (FR-43).
- Step 2 — authentication fails: no refund is possible through that session, and the attempt is logged (FR-40, NFR-11).

3.3.3 Use Case #3: ManageShowtimes

Name: ManageShowtimes — Actors: TheaterAdministrator (primary) Traces to UR-9; FR-44, FR-47, FR-48.

Flow of events

1. TheaterAdministrator authenticates and enters administrator mode.
2. TTS verifies the TheaterAdministrator role and presents the administrator interface.
3. TheaterAdministrator selects one of the twenty theaters.
4. TTS displays the film currently assigned and its existing schedule.
5. TheaterAdministrator assigns a film or changes the assignment. Each theater screens one film at a time.
6. TheaterAdministrator creates, modifies, or removes showtimes — typically six to eight per day depending on runtime.
7. TTS validates showtimes account for runtime and does not overlap within the theater.
8. TTS checks each affected showing for tickets already sold.
9. TTS applies the changes, putting new showtimes on sale within the two-week window, visible immediately on both front ends.
10. TTS logs the action with administrator, action, affected records, and timestamp.

Entry conditions

1. The user holds valid TheaterAdministrator credentials. No other role may be performed in this case.
2. The films to be scheduled exist with known runtimes.

Exit conditions

1. Success: schedule updated, on sale within the window, change attributed in the audit log.
2. Failure: schedule unchanged, attempt logged.

Alternative and exceptional flows

- Step 2 — session not authenticated as TheaterAdministrator: TTS rejects and logs the attempt (FR-47).
- Step 7 — showtimes overlap given runtime: TTS rejects the schedule and identifies the conflict.
- Step 8 — affected showing has tickets sold: TTS refuses unless explicitly confirmed, then identifies every affected ticket holder (FR-48).

3.4 Classes / Objects

Specified in Section 6.3, together with the UML class diagram, now that design work has begun. This section is retained by the SRS template's structure; see Section 6.3 for the full class list, attributes, and operations.

3.5 Non-Functional Requirements

3.5.1 Performance

NFR-1 The TTS shall support at least 1,000 concurrent users across the chain, with a design target of 2,500 to absorb peak demand. (Essential) Rationale: the client estimated "at least 1000 people at once" for twenty theaters. See A.2 on the conflicting 10,000,000 figure.

NFR-2 Under NFR-1 load, browse and search shall return within 2 seconds for 95% of requests and 5 seconds for 99%; a seat hold shall complete within 1 second for 95%. The hold bound is tight because it is what prevents double-selling. (Essential)

NFR-3 An inventory change on one front end shall appear on the other within 2 seconds, matching the client's requirement that both channels show the same data live. (Essential)

NFR-4 The TTS shall measurably outperform the system it replaces, which the client calls slow. A baseline of the incumbent shall be measured before completion, so this is testable. (Essential)

3.5.2 Reliability

NFR-5 The TTS shall have a meantime between failures of no less than 30 days. (Essential)

NFR-6 The TTS shall never confirm two tickets for the same assigned seat in one showing and shall never charge without issuing a ticket. These are absolute, with no acceptable failure rate. (Essential)

NFR-7 The TTS shall degrade partially rather than totally when a dependency fails: loss of the ReviewAggregator shall not stop sales, and loss of email shall not avoid paid tickets. (Essential)

3.5.3 Availability

NFR-8 The TTS shall be available at least 99.5% of theater operating hours, which is no more than about 1 minute of unplanned downtime per day. (Essential)

NFR-9 The TTS shall confine planned maintenance to a window outside operating hours agreed with the client and should be deployable without taking the system offline for routine updates. (Essential) Rationale: the client asked for updates "as little as possible to reduce downtime".

3.5.4 Security

NFR-10 The TTS shall issue every ticket with an identifier that is unique and infeasible to forge or duplicate, so it cannot be resold — the client's anti-scalping requirement. See A.2 on the proposed NFT implementation. (Essential)

NFR-11 The TTS shall restrict showtime configuration, override, and reporting to authenticated TheaterAdministrators, and refunds to authenticated BoxOfficeAgents or TheaterAdministrators, logging every attempt, successful or not. (Essential)

NFR-12 The TTS shall not store raw card numbers, shall carry all traffic over HTTPS, and shall store account credentials irreversibly. (Essential)

NFR-13 The TTS shall resist automated bulk purchasing such that no automated agent can exceed the twenty-ticket limit, enforced server-side. (Essential)

3.5.5 Maintainability

NFR-14 The TTS shall be organized so a defect in any one feature of Section 3.2 can be fixed without modifying the others. (Essential) Rationale: the client's chief complaint is that bugs "have gotten out of control over time with new updates" — a coupling problem, and the property this requirement exists to prevent recurring.

NFR-15 The TTS shall carry automated regression tests covering every Essential requirement, runnable before any release, and shall conform to a documented coding style — the mechanism that enforces NFR-14. (Essential)

NFR-16 The TTS shall externalize customer-facing text, ticket prices, discount percentages, seat counts, the bookable window, and the hold duration as configuration. The client should not need a software release to change the price. (Desirable)

3.5.6 Portability

NFR-17 The TTS shall work on the current and immediately preceding major versions of Chrome, Safari, Firefox, and Edge, without any browser plug-in. (Essential)

NFR-18 The TTS shall render and remain fully operable from 320 to 2560 pixels wide, covering phones through kiosk displays, since the client ruled out a native app. (Essential)

3.6 Inverse Requirements

INV-1 The TTS shall not sell tickets valid outside the client's San Diego chain, nor price in any currency but U.S. dollars.

INV-2 The TTS shall not permit a customer to refund or exchange a ticket without an employee, on any channel.

INV-3 The TTS shall not require an account to browse or to buy, nor a native mobile app for any customer task.

INV-4 The TTS shall not present a seat map for a regular theater; regular admission is general admission.

INV-5 The TTS shall not sell tickets more than two weeks before showtime or more than ten minutes after it begins, nor hold inventory for an incomplete purchase beyond five minutes.

INV-6 The TTS shall not make critic data or the feedback prompt a precondition of a purchase and shall not retain raw card numbers.

3.7 Design Constraints

DC-1 Browser-based; the client explicitly excluded a native mobile app, which constrains the entire client-side architecture.

DC-2 One chain-wide database partitioned by theater, as specified by the client, rather than one per theater.

DC-3 Payment delegated to an external gateway, both for card-handling obligations and to stay in budget.

DC-4 A user-testable prototype in four months within roughly \$500,000. These jointly rule out architectures whose cost or build time exceeds the client's tolerance and are the stated basis for the rejections in A.2.

DC-5 Single time zone (U.S. Pacific) for all showtimes, booking windows, and daily log boundaries; fixed plant of twenty theaters at 150 or 75 seats, one film each, six to eight showtimes a day.

DC-6 Critic data via a licensed feed or API, not scraping, which is legally exposed and fragile. See A-3.

3.8 Logical Database Requirements

DB-1 One logical database shall serve all twenty theaters, partitioned by theater.

DB-2 It shall hold at minimum: films with runtimes; theaters with type and capacity; showings; deluxe seats; tickets with unique identifier, price, discount, delivery method, and admission status; transactions; refunds; loyalty accounts; memberships; feedback ratings.

DB-3 It shall enforce as an integrity constraint, not by application convention alone, that no two confirmed tickets reference the same seat in the same showing.

DB-4 It shall model the reservation hold as a first-class expiring state distinct from available and sold, so a hold surviving a restart still expires correctly.

DB-5 It shall retain daily transaction logs for no less than seven years for ordinary financial record-keeping, retain purchase history for the life of an account, and store payment method references rather than raw card numbers. The exact retention period requires client confirmation.

DB-6 It shall support concurrent access at NFR-1 volumes without permitting double-sale and be backed up with a recovery point objective of no more than 5 minutes.

3.9 Other Requirements

OR-1 The TTS shall keep chain-wide daily logs of ticket purchases available to the client, realized by FR-28.

OR-2 Nearby-theater discovery is explicitly deferred to a future iteration in the client's direction.

OR-3 Self-service online refund and exchange, described in a supplementary source, is not a requirement of this release. See A.2.

4. Analysis Models

Deferred. Sequence, state-transition, and data flow diagrams are scheduled for subsequent deliverables; each will include an introduction and narrative and will trace to the requirement tags in Section 3.

4.1 Sequence Diagrams

Deferred to a subsequent deliverable; not required by the Software Design rubric for Section 6. Expected coverage: the PurchaseTicket flow including the PaymentGateway exchange, and the RefundTicket flow.

4.2 State-Transition Diagrams (STD)

Deferred to a subsequent deliverable; not required by the Software Design rubric for Section 6. Expected coverage: the lifecycle of a seat through available, held, sold, admitted, and refunded — the state model NFR-8 depends on.

4.3 Data Flow Diagrams (DFD)

Deferred to the Architecture Design with Data Management deliverable.

5. Change Management Process

This SRS is a baseline, not a fixed document. Changes may be submitted by the client, by any member of the engineering team (typically on finding a requirement ambiguous, contradictory, or infeasible), or by the instructor as a stakeholder.

A change request is submitted in writing to the project owner stating the requirement tags affected, the change, the reason, and its assessed impact on budget, schedule, and other requirements. Requests arising in conversation are transcribed into this form and confirmed back to the client, so no requirement enters the baseline on an unrecorded conversation.

1. The project owner logs the request and traces the affected tags forward into dependent design, test, and implementation artifacts.
2. The team assesses it against the constraints of Section 2.4 — principally the \$500,000 budget and four-month deadline.
3. Anything affecting scope, cost, or schedule needs the client's written approval; a correction of an ambiguity or error may be approved by the project owner alone.
4. Anything invalidating an assumption in Section 2.5 is escalated to the client explicitly. A-4 on concurrency is the clearest case were the client to confirm 10,000,000 concurrent users, the correct response would be to rewrite this document and renegotiate the project, not amend it.
5. On approval the text is amended in place, keeping its tag if the meaning is refined and taking a new tag if it is genuinely new. Tags are never reused; withdrawn requirements are marked withdrawn, not deleted, so artifacts tracing to them stay interpretable. The version is incremented, the Revision History updated, and the document recirculated.

6. Software Design

This section is the Software Design Specification for the Theater Ticketing System, added as its own section per the course's build-up-one-document format. Sections 1 through 5 above are unchanged from the Requirements Specification submitted July 14, 2026; this section describes how the system specified there will be built, at the level of software architecture and a UML class design. It is written for the developers who implement and maintain the TTS, not for the client — where Section 1 explained what the system does in the client's terms, this section explains how it is put together in the engineering team's terms.

6.1 System Overview

The Theater Ticketing System is a three-tier web application: browser-based clients (customer web, in-theater kiosk, and box office/admin) talk to a single application server over HTTPS, and the application server is the only component that talks to the chain-wide database or to any external system. No client talks directly to the database or to the PaymentGateway, ReviewAggregator, or email service — every one of those relationships is mediated by the application server, which keeps the correctness guarantees of Section 3.5 (in particular NFR-6, no double-sold seat and no charge without a ticket) enforceable in one place rather than replicated across three front ends.

The design follows directly from the constraints already fixed in Section 2.4 and Section 3.7: DC-1 (browser-based, no native app) rules out anything but a web client; DC-2 (one database partitioned by theater, not one per theater) rules out a per-theater deployment; DC-3 (payment delegated to a gateway) keeps card handling out of the TTS entirely; and DC-4 (four months, \$500,000) rules out a distributed microservice architecture in favor of a single application server organized internally into services — the NFR-14 maintainability requirement (a defect in one feature must be fixable without touching the others) is met by that internal service boundary, not by physically separate deployments, which the budget and timeline do not support.

6.2 Software Architecture

6.2.1 Architecture Diagram

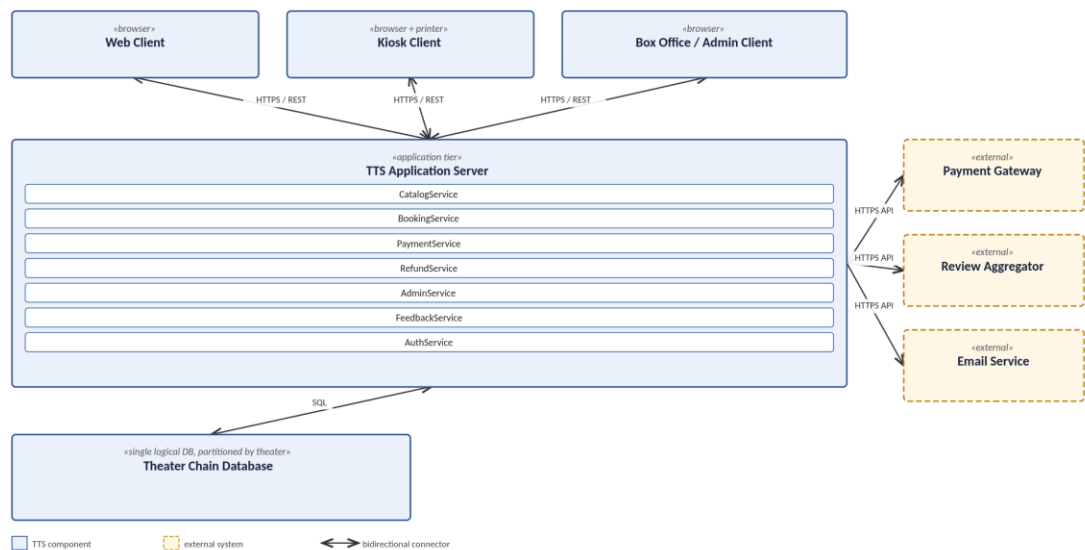


Figure 2. Theater Ticketing System — Software Architecture

6.2.2 Component Descriptions

Component	Description	Connects to
Web Client	The customer-facing browser application. Realizes FR-1 and FR-2's public front end. No native app, per DC-1.	TTS Application Server (HTTPS/REST)
Kiosk Client	The same customer-facing application running on in-theater kiosk hardware, plus the touch input and ticket-printer interface required by FR-6.	TTS Application Server (HTTPS/REST)
Box Office / Admin Client	The employee-facing interface: refunds (FR-40–FR-43) for a BoxOfficeAgent, and showtime, override, and reporting (FR-44–FR-48) for a TheaterAdministrator. Reachable only by an authenticated session (FR-4).	TTS Application Server (HTTPS/REST)

Component	Description	Connects to
TTS Application Server	The single application tier. All business logic, all validation, and all correctness guarantees (NFR-6, NFR-7) live here, organized into the seven services described below, so that no client can bypass a rule by calling the database directly.	All three clients; Database; PaymentGateway; ReviewAggregator; Email Service
Theater Chain Database	The single logical database, partitioned by theater, specified in DB-1 through DB-6. Holds films, theaters, showings, seats, reservations, tickets, transactions, accounts, memberships, feedback, and the audit log.	TTS Application Server only (SQL)
Payment Gateway	External. Authorizes and captures payment, and processes reversals. The TTS never stores a raw card number (NFR-12, INV-6); this is the only component that ever sees one.	TTS Application Server (HTTPS API)
Review Aggregator	External. Supplies critic scores and quotes (FR-16). Its unavailability must never block a sale (FR-18), so the Application Server treats it as a best-effort, cached dependency rather than one on the purchase critical path.	TTS Application Server (HTTPS API)
Email Service	External. Delivers tickets and confirmations (FR-11). Its failure must never void a paid ticket (FR-39, FR-53), so ticket confirmation does not block on delivery succeeding.	TTS Application Server (HTTPS API)

The seven services inside the TTS Application Server each own one feature area from Section 3.2, which is the internal boundary NFR-14 depends on:

- CatalogService — showtime browsing and search (FR-14 through FR-19); owns the read path to Film, Theater, and Showing.
- BookingService — ticket purchase and seat reservation (FR-20 through FR-26); owns Reservation and Seat, and is the only service that is called Showing.holdInventory().
- PaymentService — payment processing and discounts (FR-27 through FR-32); owns Transaction and is the only service that calls out to the PaymentGateway.
- RefundService — refunds (FR-40 through FR-43); owns the refund path on Ticket and Transaction, callable only by an authenticated BoxOfficeAgent or TheaterAdministrator session.
- AdminService — administrator mode (FR-44 through FR-48); owns TheaterAdministrator's operations and the audit log.
- FeedbackService — customer feedback (FR-49 through FR-50) and bot mitigation/demand queueing (FR-51 through FR-53); grouped together because both are auxiliary to the purchase path rather than on it.
- AuthService — authentication for all three roles (Employee.authenticate(), and Moviegoer account login under FR-24, FR-33–FR-35); the single code path NFR-18's audit logging depends on.

6.3 UML Class Diagram

6.3.1 Class Diagram

Figure 3 shows the fifteen classes of the domain design: candidate entities identified in Section 3.4 of the Requirements Specification, refined here into concrete attributes and operations. Visibility follows UML convention (+ public, - private, # protected); PaymentGateway and ReviewAggregator are drawn with a dashed outline to mark them as external interfaces rather than classes the TTS implements.

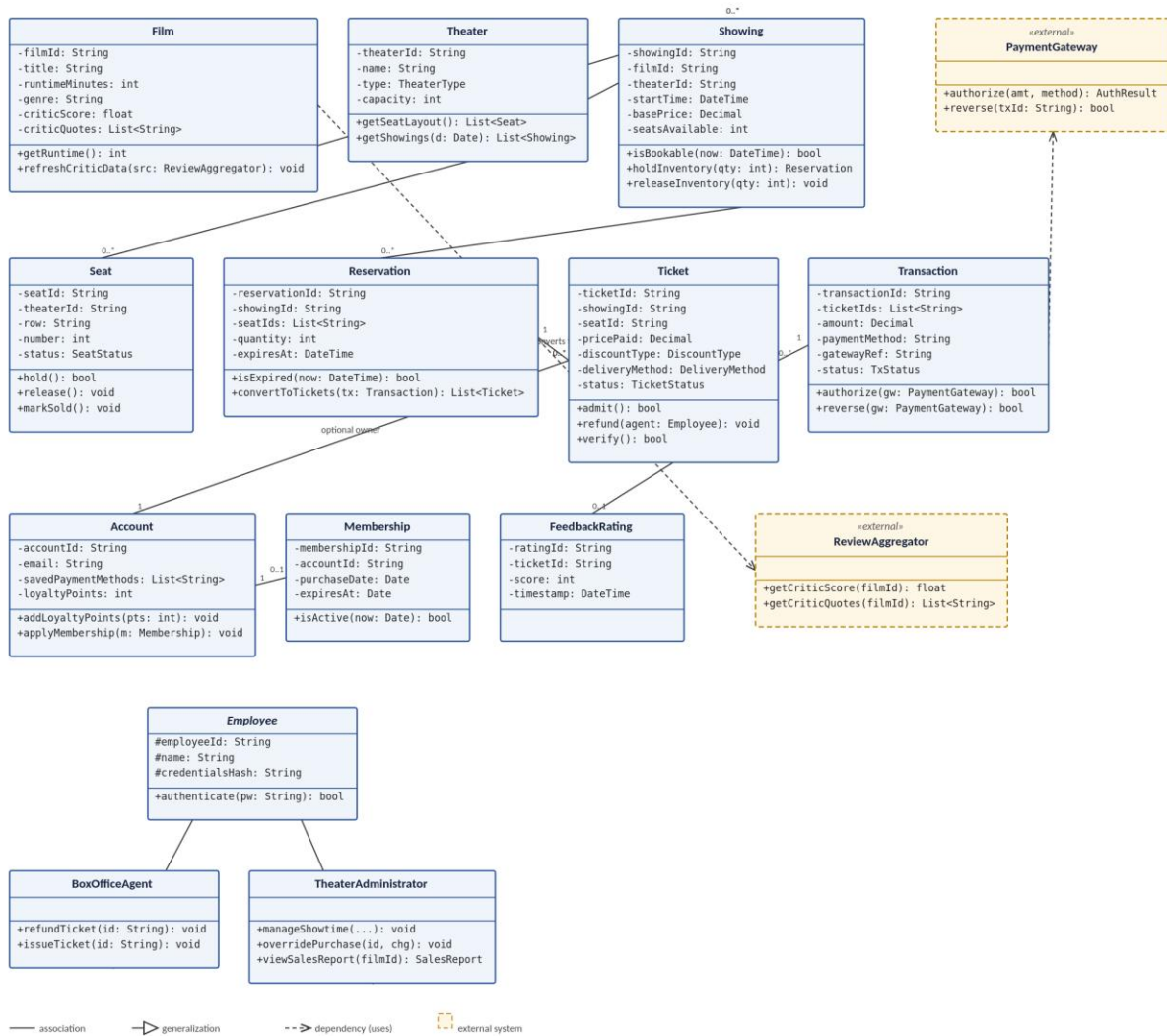


Figure 3. Theater Ticketing System — UML Class Diagram

6.3.2 Class Descriptions

Each class below is described with its role in the design, followed by every attribute and operation shown in Figure 3, in the same order, with the datatype or function signature repeated from the diagram and a one-line note on its purpose. Visibility follows UML convention: + public, - private, # protected.

6.3.2.1 Catalog and Scheduling

Film, Theater, and Showing model what the client is selling: a film, in a theater, at a time.

Film

Member	Purpose
-filmId: String	Unique identifier for the film record.
-title: String	Display title shown to Moviegoers (FR-15).
-runtimeMinutes: int	Used when validating that showtimes do not overlap (see ManageShowtimes flow step 7, Section 3.3.3).
-genre: String	Supports future filtering; not required by a specific FR.
-criticScore: float	Aggregated score from the ReviewAggregator, displayed per FR-16.
-criticQuotes: List<String>	Representative critic quotes displayed per FR-16.
+getRuntime(): int	Read accessor used by ManageShowtimes to compute non-overlapping schedules.
+refreshCriticData(src: ReviewAggregator): void	Pulls current score and quotes from the ReviewAggregator on a schedule, since reviews are not fetched per browse request. Realizes FR-16 and the degrade-safely behavior of FR-18.

Theater

Member	Purpose
-theaterId: String	Identifies one of the twenty theaters.
-name: String	Display name.
-type: TheaterType	REGULAR or DELUXE; drives the FR-22/FR-23 branch in PurchaseTicket.
-capacity: int	150 (regular) or 75 (deluxe), per DC-5.
+getSeatLayout(): List<Seat>	Returns the seat map for a deluxe theater; a regular theater returns an empty list, since INV-6 forbids presenting a seat map for general admission.
+getShowings(d: Date): List<Showing>	Supports showtime browsing and search (FR-14).

Showing

Member	Purpose
-showingId: String	Unique identifier for the showing.
-filmId: String	Foreign key to the scheduled film.
-theaterId: String	Foreign key to the theater it screens in.
-startTime: DateTime	The showtime.
-basePrice: Decimal	Pre-discount ticket price; FR-5 requires this to display identically everywhere.
-seatsAvailable: int	Live availability count; the field NFR-4's 2-second cross-channel update applies to.
+isBookable(now: DateTime): bool	Enforces the two-week/ten-minute booking window (FR-19, INV-5).
+holdInventory(qty: int): Reservation	Creates a Reservation and atomically decrements availability. This is the operation NFR-6's no-double-sell guarantee ultimately rests on.
+releaseInventory(qty: int): void	Called on hold expiry or an abandoned purchase (FR-25).

6.3.2.2 Inventory and Sale

Seat, Reservation, Ticket, and Transaction model the path from an available seat to a confirmed, paid, admissible ticket — the path most of Section 3.2's error-handling requirements protect.

Seat

Member	Purpose
-seatId: String	Unique identifier for a deluxe-theater seat.
-theaterId: String	Foreign key to the owning theater.
-row: String	Row label shown on the seat map.
-number: int	Seat number within the row.
-status: SeatStatus	AVAILABLE, HELD, or SOLD.
+hold(): bool	Atomic compare-and-set from AVAILABLE to HELD; returns false if another Moviegoer already claimed it. This is the concrete mechanism behind FR-29 and NFR-6.
+release(): void	Returns a seat to AVAILABLE on hold expiry or refund.
+markSold(): void	Transitions HELD to SOLD on confirmed payment.

Reservation

Member	Purpose
-reservationId: String	Unique identifier for the hold.
-showingId: String	The showing being reserved against.
-seatIds: List<String>	Held seats, for a deluxe theater; empty for general admission.
-quantity: int	Ticket count, up to twenty (FR-20).
-expiresAt: DateTime	Five-minute hold deadline (FR-25).
+isExpired(now: DateTime): bool	Checked on every access and by a background sweep that releases lapsed holds.

Member	Purpose
+convertToTickets(tx: Transaction): List<Ticket>	Invoked only on successful payment; the single code path that mints tickets, which is what keeps FR-38's \"never charged without a ticket\" guarantee enforceable in one place rather than several.

Ticket

Member	Purpose
-ticketId: String	The unique, non-forgable identifier required by NFR-15.
-showingId: String	The showing this ticket admits to.
-seatId: String	Assigned seat; null for general admission.
-pricePaid: Decimal	Final price after any discount.
-discountType: DiscountType	NONE, STUDENT, MILITARY, or SENIOR (FR-39).
-deliveryMethod: DeliveryMethod	EMAIL or BOX_OFFICE (FR-50).
-status: TicketStatus	CONFIRMED, ADMITTED, or REFUNDED.
+admit(): bool	Transitions CONFIRMED to ADMITTED; returns false and does not admit if the ticket is already ADMITTED or REFUNDED (FR-38, FR-42).
+refund(agent: Employee): void	Only meaningful when called with a BoxOfficeAgent or TheaterAdministrator; FR-40's role restriction is enforced by AuthService before this is ever reached.
+verify(): bool	Checks identifier authenticity independent of admit(), used by the door-scanning step of PurchaseTicket's exit flow.

Transaction

Member	Purpose
-transactionId: String	Unique identifier for the payment attempt.
-ticketIds: List<String>	Tickets this transaction paid for.
-amount: Decimal	Total charged, in U.S. dollars (FR-34).
-paymentMethod: String	Credit card or online payment service (FR-32).
-gatewayRef: String	The PaymentGateway's own reference, used to look up a transaction if a reversal must be retried.
-status: TxStatus	AUTHORIZED, CAPTURED, or REVERSED.
+authorize(gw: PaymentGateway): bool	Calls out to the gateway; on false, Reservation.convertToTickets is never invoked (the FR-29/FR-37 decline and unreachable-gateway paths).
+reverse(gw: PaymentGateway): bool	Used by both RefundTicket and the automatic FR-38 reversal when confirmation fails after authorization.

6.3.2.3 Customer Engagement

Account, Membership, and FeedbackRating are all optional from the Moviegoer's side — none is a precondition of a purchase, per INV-3 and INV-6.

Account

Member	Purpose
-accountId: String	Unique identifier for an optional loyalty account.
-email: String	Contact address, also used for ticket delivery when elected.
-savedPaymentMethods: List<String>	References to saved payment methods — never raw card numbers (INV-6, DB-5).
-loyaltyPoints: int	Accrued balance (FR-46).
+addLoyaltyPoints(pts: int): void	Called after a completed purchase made under an authenticated account.
+applyMembership(m: Membership): void	Links a purchased membership card to the account (FR-45).

Membership

Member	Purpose
-membershipId: String	Unique identifier for the subscription.

Member	Purpose
-accountId: String	The account it belongs to.
-purchaseDate: Date	When the membership was bought.
-expiresAt: Date	When it lapses.
+isActive(now: Date): bool	Checked when a Moviegoer applies a membership during PurchaseTicket.

FeedbackRating

Member	Purpose
-ratingId: String	Unique identifier for the rating.
-ticketId: String	The purchase it was collected against.
-score: int	The selected smiley-face value, 1 to 5.
-timestamp: DateTime	When the rating was recorded.

6.3.2.4 Staff Roles

Employee is an abstract base; the TTS never instantiates it directly, only its two subclasses. Both extend it rather than duplicating authenticate(), which is why NFR-18's logging requirement can be satisfied in one place.

Employee (abstract)

Member	Purpose
#employeeId: String	Unique identifier, visible to subclasses only (protected).
#name: String	Employee name, for audit-log attribution (NFR-18).
#credentialsHash: String	Irreversibly stored credential (NFR-12); never a plaintext password.
+authenticate(pw: String): bool	Shared by both subclasses. Kept on the abstract base — not duplicated on each role — so every authentication attempt, successful or not, is logged through one code path, which is what NFR-18's blanket logging requirement depends on.

BoxOfficeAgent

Member	Purpose
+refundTicket(id: String): void	Realizes FR-40 through FR-43; delegates to Ticket.refund().
+issueTicket(id: String): void	Hands over a box-office-collection ticket (FR-50).

TheaterAdministrator

Member	Purpose
+manageShowtime(...): void	Realizes FR-44 — create, modify, remove showtimes and assign films to theaters.
+overridePurchase(id, chg): void	Realizes FR-45.
+viewSalesReport(filmId): SalesReport	Realizes FR-46.

6.3.2.5 External Interfaces

PaymentGateway and ReviewAggregator are contracts the TTS depends on, not classes it implements — shown with a dashed outline in Figure 3 to mark them as external, matching their treatment in Section 2.1.

PaymentGateway «external»

Member	Purpose
+authorize(amt, method): AuthResult	The external authorization calls every Transaction.authorize() wraps.
+reverse(txId: String): bool	The external reversal calls every Transaction.reverse() wraps.

ReviewAggregator «external»

Member	Purpose
+getCriticScore(filmId): float	Backs Film.refreshCriticData().
+getCriticQuotes(filmId): List<String>	Backs Film.refreshCriticData().

6.4 Development Plan and Timeline

6.4.1 Task Partitioning and Timeline

The plan below spans the four-month, sixteen-week prototype window fixed by DC-4, organized around the seven services of Section 6.2.2 so that each phase produces a working, testable slice rather than a layer that only becomes useful once every other layer is also done.

Weeks	Phase	Tasks	Owner
1–2	Foundation	Stand up the database from Section 3.8/DB-1–DB-6; project scaffolding, CI pipeline, and the coding-standards document NFR-15 requires.	Bollampally
3–6	Browse and book	CatalogService and BookingService: showtime browsing (FR-14–FR-19), seat reservation and the five-minute hold (FR-20–FR-26), and AuthService for account login (FR-24, FR-33–FR-35).	Bollampally
7–9	Pay and refund	PaymentService (FR-27–FR-32) integrated against the PaymentGateway sandbox, ticket issuance and validation (FR-36–FR-39), and RefundService (FR-40–FR-43).	Bollampally
10–12	Administer and protect	AdminService (FR-44–FR-48) and FeedbackService, including bot mitigation and demand queueing (FR-51–FR-53).	Bollampally
13–14	Kiosk and hardening	Kiosk client and printer integration (FR-6), cross-browser pass against NFR-17–NFR-18, and the security requirements of NFR-10–NFR-13.	Bollampally
15–16	Verify and hand off	Regression suite to NFR-15, load test against NFR-1–NFR-3, incumbent-system baseline measurement for NFR-4, client UAT, and prototype handoff.	Bollampally

6.4.2 Team Member Responsibilities

Group CS250-26 has one member, Pranith Bollampally, who owns every task above and every service in Section 6.2.2. The phase breakdown in 6.4.1 stands in for the task-partitioning table a multi-person group would use to divide work by teammate; here it divides work by time and by service boundary instead, so that the sequencing itself — database first, then the purchase path, then administration, then hardening — is the record of how the work is planned to proceed.

A. Appendices

The information in these appendices is not part of the requirements baseline. It is recorded to make Section 3 interpretable — in particular, to show where each requirement came from and, where the sources disagreed, why this document says what it says.

A.1 Elicitation Sources

- Client interview — a structured 25-question interview covering channel, budget, ticket types, discounts, scale, the incumbent's deficiencies, seating, refunds, administrator capability, and reviews. The most authoritative source: the client speaking directly, and internally consistent.
- Client-supplied requirements list — nine functional and two non-functional requirements, consistent with the interview.
- Supplementary elicitation notes and requirements list — contribute requirements found nowhere else (languages, payment breadth, loyalty accounts, queueing), carried into Section 3, but also conflict with the interview in eight places, resolved in A.2.

Where sources agree, this document follows them. Where they disagree, the client interview prevails, because it is the client's own account of their own business and its figures are mutually consistent — twenty theaters, one city, \$500,000, four months. A supplementary requirement is carried when it does not contradict that account, and rejected or flagged where it does.

A.2 Requirement Conflicts and Their Resolution

The sources contradict one another in eight places. The first is material enough that the project should not proceed without client confirmation of it.

Conflict	Interview says	Supplementary says	Resolution
Concurrency	"At least 1000 at once" across 20 theaters	10,000,000 concurrent users	Interview prevails (NFR-1: 1,000, 2,500 headroom). 10M is roughly 3x San Diego County buying tickets to 20 theaters at once — irreconcilable with \$500K/4 months. Treated as an error (A-4). If confirmed, this SRS is void, not amendable.
Seat selection	Deluxe assigned; regular general admission	Users select specific seats	Interview prevails (FR-20, FR-21 split by type). Supplementary source generalizes the deluxe case to all theaters incorrectly.
Refunds	In-person employee only	Self-service return/exchange with account	Interview prevails (FR-40, INV-2). Self-service recorded out of scope (OR-3).
Ticket uniqueness	Unique, non-replicable (anti-scalping)	Each ticket is an NFT	Requirement accepted, implementation rejected. NFR-10 states the underlying property without mandating a blockchain, which is a design decision, not a requirement, and costs more than the budget supports.
Currency	San Diego only, one time zone	Converted to local currency	Interview prevails (INV-1). A single-city chain has one currency; source appears to describe a different, international system.
Discounts	Student, military, senior	Student/veteran, weekdays only	Union taken (FR-31): all three categories, no weekday restriction, since the interview's phrasing is unrestricted. Weekday limit flagged for confirmation.
Payment methods	Not addressed	Card, PayPal, Bitcoin	Partially carried (FR-27: card, online payment). Bitcoin dropped — unmentioned by client, costly, and appears alongside the NFT item from the same source.
Languages	Not addressed	English, Spanish, Swedish	Carried with a flag (FR-3). English/Spanish unarguable for San Diego; Swedish stated twice but anomalous for a San Diego-only chain — recorded as A-6 for confirmation.

The concurrency and NFT resolutions are the ones most likely to be wrong and worth restating. On concurrency: this document assumes 1,000 is right and 10,000,000 is a mistake, and the entire cost basis of the project rests on that; it is the highest-impact open question here. On the NFT: what the client asked for — tickets that cannot be copied or scalped — is captured in full as NFR-10, but the blockchain itself was not, since a specific technology does not belong in a requirements document and can be added back if the client has an independent reason to want it.